Assignment 11
Inheritance and Polymorphism

Problem Statement

Develop a system that reads shape data from an input file, constructs appropriate shape objects based on the provided structure, and provides both textual and graphical representations of these shapes. The system should recognize different geometric figures such as points, lines, quadrilaterals, circles, squares, rectangles, parallelograms, trapeziums, spheres, cubes, cones, and cylinders. The program must dynamically store these shapes and allow users to view their properties and graphical representations interactively.

System Functionality

1. Shape Properties Calculation

- The system first displays properties of each shape, such as area, perimeter, or volume, depending on the shape type.
    - Length for 1D-Shapes
    - Perimeter and Area of 2D-Shapes
    - Volume (and Surface-Area(optional)) for 3D-Shapes
- You can create helper functions for calculations and add required attributes in the ADTs according to the shape.
- Users press a key to proceed to the next shape's properties.

2. Interactive Shape Visualization using Raylib

- After displaying properties, the system enters an interactive mode where each shape is graphically rendered using Raylib.
- Users can navigate to the next shape using the **Enter** key.
- Shapes are drawn on a graphical window initialized using Raylib, with appropriate dimensions and colors.

Implementation Requirements

- Use **abstract classes** and **virtual functions** appropriately.
- Remember each virtual function of a parent class must be part of the child class
- **Sample Class Hierarchy:**
    - Shape

- 1D
    - Line,
    - Segment,
    - Ray
    - others
- 2D
    - Polygone
        - Quadrilateral
            - **Square**
            - **Rectangle**
            - **Parallelogram**
            - others
        - Triangle
        - Octagon
        - others
    - Circle
    - others
- 3D
    - Sphere
    - Cube
    - others

Input Format

- The first line contains an integer **N**, representing the number of shapes in the dataset.
- Each subsequent line contains shape-specific data:
    - **Type Identifier** (Integer) indicating the shape type (e.g., 1 for 2D Point, etc.).
    - **Coordinate values** defining the shape's dimensions.
    - **Additional parameters**, such as radius or height, for applicable shapes.
- The file follows a strict format, where each shape type has a predefined structure.

Example Input

```
11
1 2 3
2 5 5 3 3
3 2 2 3 3 5 5 6 6
4 2 2 5
5 0 0 5 5 0 5 5 0
6 0 0 7 0 0 3 7 3
7 0 1 3 0 2 -1 5 -2
8 0 0 7 0 2 5 5 5
9 3 5 5 6
10 0 0 0 6 10
11 5 5 5 10 5
```

Shape Representations

1. **2D Point** (x, y)
2. **Line** (x1, y1, x2, y2)
3. **Quadrilateral** (x1, y1, x2, y2, x3, y3, x4, y4)
4. **Circle** (center_x, center_y, radius)
5. **Square** (four corner points)
6. **Rectangle** (four corner points)
7. **Parallelogram** (four corner points)
8. **Trapezium** (four corner points)
9. **Sphere** (center_x, center_y, center_z, radius)
10. **Cone** (base center_x, base center_y, base center_z, radius, height)
11. **Cylinder** (base center_x, base center_y, base center_z, radius, height)

Expected Output

Console Output (Example for Circle)

```
Circle Properties:
Radius: 50
Area: 7853.975
Circumference: 314.159
Press any key to continue...
```

Graphical Output (Raylib window displays a circle, followed by other shapes as the user presses **Enter**).

Sample Main

(You can change this if needed)

```
int main() {
    vector<Shape*> shapes = LoadShapesFromFile("shapes.txt");

   // Display shape properties first
    for (Shape* shape : shapes) {
        shape->PrintProperties();
        cout << "Press any key to continue..." << endl;
        _getch();
    }
    // Initialize Raylib window
    const int screenWidth = 800;
    const int screenHeight = 600;
    InitWindow(screenWidth, screenHeight, "Raylib Shapes Visualization");
    SetTargetFPS(60);
    int currentShapeIndex = 0;
    while (!WindowShouldClose() && currentShapeIndex < shapes.size()) {
        BeginDrawing();
```

```
        ClearBackground(RAYWHITE);
        shapes[currentShapeIndex]->Draw();
        EndDrawing();
        if (IsKeyPressed(KEY_ENTER)) {
            currentShapeIndex++;
        }
    }
 // Cleanup
    CloseWindow();
    for (Shape* shape : shapes) {
        delete shape;
    }

    return 0;
}
```

Only for those who opened it at the last movement